A SURVEY OF ADAPTIVE BEHAVIOR TREES BY

Lauren Gayle for the Robotics in the Real World REU in Robotics presented on August 14, 2023.

Title: Survey of Adaptive Behavior Trees

Abstract approved: _____

Geoffrey A. Hollinger and Emily Rose Scheide

A behavior tree that is adaptive is one whose structure can change in some way to accommodate for unpredictable environments. This paper surveys adaptive behavior trees, looking at the different applications where adaptive behavior trees are being used, the different methods through which a behavior tree is made to be adaptive, and the current gaps in literature regarding adaptive behavior trees.

Survey of Adaptive Behavior Trees

by
Lauren Gayle

A SURVEY OF ADAPTIVE BEHAVIOR TREES

submitted to

Oregon State University

in partial fulfillment of
the requirements for the
program of

Robotics in the Real World REU

Presented August 14, 2023

A survey for Robotics in the Real World REU on adaptive behavior trees by Lauren Gayle presented on August 14, 2023.

APPROVED:

Major Professor, representing Robotics

Associate Dean of Graduate Studies of the College of Engineering

---

Dean of the Graduate School

I understand that my survey of adaptive behavior trees will become part of the permanent collection of Oregon State University libraries. My signature below authorizes release of my survey of adaptive behavior trees to any reader upon request.

Lauren Gayle, Author

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1: Introduction

Behavior trees (BTs) have garnered a large amount of attention in various robotic domains within the last decade. Originally starting as a tool for more dynamic NPC behavior in computer games, BTs have since been applied to many fields within robotics such as autonomous vehicles, aerial robotics, robotic manipulators, and a myriad of other applications as seen in [14]. BTs obtained such popularity from the unique benefits they offer in comparison to other control architectures such as Finite State Machines. In a survey of BTs by Iovino et al., they talk on how BTs provide reactivity to actions, allowing for the robot to stop before an action is complete and go into another, more necessary action. The usage of BTs to determine a robot's actions is extremely useful in environments that are somewhat unpredictable or change quickly, as the BT can simply switch to another action that is more fitting for its current environment. The benefit of BTs in robotics is seen in applications such as Human-Robot interactions, for example in [7] and [12], a BT is used to determine the actions of a robot interacting with children. [5] talks on how BTs are beneficial in multi-robot systems due to their ability to run tasks in parallel and the fault-tolerant properties of the fallback nodes. [28] uses BTs to make a path planning method that can be used by multiple tracking Unmanned Aerial Vehicles to navigate complex environments. Overall, BTs have been growing in popularity due to how reactive and modular they are, making them a good control architecture for navigating changing environments.

As BTs become more prevalent in various sectors of robotics, there comes a need to make behavior trees even more reactive for unpredictable, unknown, and dynamic environments. This is where adaptive behavior trees (ABTs) come into play. Where a standard BT can easily switch between predetermined actions based on specified conditions, BTs that are adaptive can change in structure to fit the environment. The manner in which these changes happen varies between the different approaches used to create an ABT, but they all have the general purpose of making a BT that can reorder tasks, implement new tasks at the falilure of one, and/or change in overall structure as new information is acquired by the robot about the surrounding environment.

This paper surveys the current applications of ABTs, the different methods through which BTs are made to be adaptive, and where within the general domain of robotics ABTs have yet to be utilized. An illustration of the topics covered in the paper is seen in Fig. 1.1. The main objective of this paper is to provide an overview of the current uses of ABTs, how they are being created, and identify the gaps in the literature on ABTs.

In section 2, the paper looks at the various applications of ABTs,in the robotics fields of mobile robotics, manipulators, human-robot interactions, as well as a an applications somewhat outside of the robotics in the domain of AI agents. This section seeks to give a broad overview of the different ways ABTs have been used and display the versatility of ABTs. Section 3 goes into how ABTs are made to be adaptive, covering the various methodology such as synthesizing and updating a BT from a task planner, utilizing reinforcement learning, using linear temporal logic, expanding a BT iteratively, and combining BTs with active inference. Section 4 takes a look at the applications of ABTs and methodology for creating ABTs, and points out the gaps in the literature

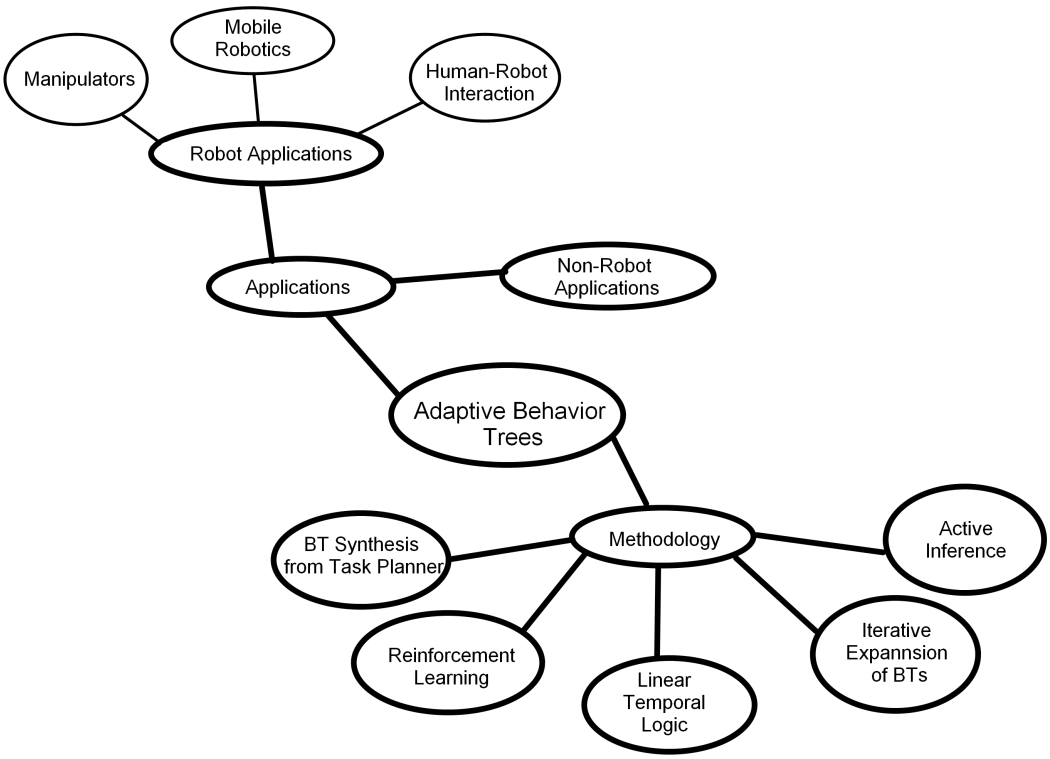

Figure 1.1: Overview of survey topics

being surveyed. Section 5 concludes the paper, giving a general summary of what has been covered.

1.1 A Brief Overview of Behavior Trees

Behavior trees are a control architecture that, unlike their counterparts of finite state machines and decision trees, are built in regards to tasks rather than states. This difference allows for BTs to be more readable, recursive, and modular [12]. BTs are direct rooted trees made up of the control flow nodes and execution nodes [12]. The control

Node type	Symbol	Succeeds	Fails	Running
Fallback	?	If one child succeeds	If all children fail	If one child returns Running
Sequence	$\rightarrow$	If all children succeed	If one child fails	If one child returns Running
Parallel	$\rightrightarrows$	If $\geq M$ children succeed	If $> N - M$ children fail	else
Action	text	Upon completion	If impossible to complete	During completion
Condition	text	If true	If false	Never
Decorator	$\Diamond$	Custom	Custom	Custom

Figure 1.2: Behavior Tree Nodes

flow nodes are the sequence node, the fallback node, the parallel node, and the decorator node.The execution nodes are the action nodes and condition nodes. All nodes return either success or failure, and all sans the condition nodes return running [12]. The table in 1.2 from [6] shows under what conditions each node succeeds, fails, and runs, as well as the symbols that typically represent them. Control flow nodes guide the flow of the depth-first search through the tree via ticking, and execution nodes wither incite an action or check for a condition to be fulfilled.

Chapter 2: Applications

Behavior Trees have had various applications, starting off as a means to create more dynamic NPCs in video games, they have gained traction all throughout the robotics industry. They are in several different fields already as seen in [14], and as seen in table 2.1 ABTs are garnering traction in some of the robotic fields as well, from robotic manipulators to human-robot interaction, as well as non-robotic applications such as AI agents.

Applications	Papers
Manipulators	[4] [8] [15] [18] [23] [24]
Mobile Robotics	[11] [13] [17] [26] [29]
Human-Robot Interactions	[2]
AI Agents	[22]

Table 2.1: Applications of Adaptive Behavior Trees.

2.1 Robotics

ABTs have a multitude of applications within the domain of robotics. They are a powerful, reactive control architecture tool that has been applied to a multitude of different fields within the robotics domain. ABTs provide the ability to account for unexpected or changing environmental factors that various robots can encounter, allowing for them to continue to operate autonomously within these dynamic environments.

2.1.1 Manipulators

Behavior trees have been used for robotic manipulators since around 2012 according to [14]. ABTs for robotic manipulation tasks allows for more reactivity than a static BT would as it allows for the adjustment of tasks in the event of new variables being introduced.

Many of the papers that apply their ABT to robotic manipulators have the a robotic arm of some kind either tied to a mobile or stationary base, and this arm must move objects in some specified manner. In [8], their ABT method is validated using a simulated robotic arm that has to sort 3 colored boxes in a scenario where their starting location is known and one where it is not. They compare the performance of their ABT to that of a standard BT, and in both case studies the ABT performed better in terms of tree complexity and computational time.

Many applications of ABTs for mobile manipulators have the manipulator moving objects of interest from one spot to another with environmental factors changing where the objects are or blocking the manipulator's ability to get to the goal location. The manipulator in these applications require fully observable environments so they can detect and adjust immediately to any environmental changes. The simulated TIAGo mobile manipulator and the dual-armed Yumi are used in [24] to move objects from zone to zone on a board, with the environmental changes being that the objects of interest are not always in their specified location, and the manipulators are equipped with a camera that fully observes the board. In [4], their ABT is applied to a simulated mobile manipulator that has to get an object to a goal, avoiding path obstructions and external agents

moving through the scene, and a simulated dual-armed manipulator that as to assemble a cellphone with parts scattered randomly across a table, and certain parts must be placed in a specific order and orientation. These applications all use fully observable environments for the manipulator to function within, allowing for the manipulator to immediately see the environmental changes and the ABT can account for and adjust to these changes.

Additionally, ABTs also give the ability for robotic manipulators to perform well in partially observable environments. In [18], they use a simulated mobile manipulator placed in front of a cupboard with a light inside, and a stick it must grab. The ability for the manipulator to grab the stick is dependent on if the light is on or off and the ability for the manipulator to turn off and on the light is dependent on if the door is open or closed. The ABT allows for the manipulator to actively replan tasks despite its limited perception of its environment. [23] similarly uses ABTs to overcome partially observable environments, as their ABT is applied to a mobile manipulator that has to move a crate from one location to another, but cannot perceive if the crate is at the table until the robot is at the table. This paper applies this to a physical TIAGo mobile manipulator and has it stock shelves in a store mockup. This ABT has online changes based on the mobile manipulator's partially observable environment, with a specific focus on using ABTs for robust task execution and using ABTs to deal with noisy sensor readings. [15] applies their ABT to a mobile manipulator with a similar task of moving objects from one place to a goal location, with a camera that can face up when moving and must face down when manipulating objects. All of these applications utilize ABTs within partially observable environments, and the ABTs allow for the tasks to be executed despite lim-

ited perception as they can account for and adjust available tasks to still complete the main objective of the manipulators.

2.1.2 Mobile Robotics

Mobile robotics refers to robots that can physically move about their environments. The types of robots that fall within the mobile robotics category range from unmanned aerial vehicles (UAVs) to delivery robots. BTs have been adapted by this field of robotics because as robotics becomes more and more prevalent, they will be encountering more unknown and unpredictable variables that need to be accounted for. ABTs allow for mobile robots to be reactive, and handle the changes in the robot's surroundings as it navigates dynamic environments.

[17] uses ABTs as a means to control micro aerial vehicles (MAVs). The goal of the work of Lan et al. is to develop a task and acting framework for autonomous MAVs using a BT interface that can execute a plan that changes based on environmental variables. [17] uses a simulation to display the framework they developed, and in this simulation, a MAV executes a surveillance task around points A, B, and C to search for a target.It searches each point and reports to one of its three uploading sites when the target is found. It re-plans while executing the tasks when it finds the target while going from one point to another, and also re-plans its tasks when it fails to upload the target information to the nearest uploading site so that it goes to the second one. This is illustrated in Fig. 2.1. The ABT is dependent on a fully observable environment, as the MAV has to know where each point is, the location of each uploading site, and be able to see and

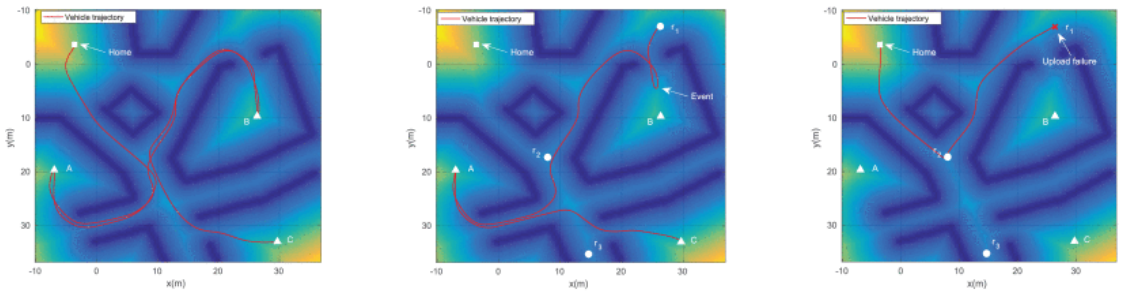

Figure 2.1: Paths of MAV

(a) Shows the MAV traverse three sites in search for its target. In (b), a target is found in an unexpected location, triggering the re-planning process, and causing the MAV to fly to site r1 and upload its information. (c) Shows an unsuccessful uploading action, causing the BT to modify the task-level model and trigger a re-planning process again. The MAV then flies to site r2, uploads its information, and flies home [17]

interpret the environment in between those locations.

[26] demonstrates their ABT in a simulated environment of factory workstations. The robot's objective is to navigate through the factory from various workstations and the storehouse to obtain pieces and place them in other locations. To demonstrate the adaptability of the robot, random events were added to the simulation such as blocks in paths, broken workstations, and the sudden loss of a lot of battery. This usage of an ABT to traverse a factory and complete simple tasks in an environment that is partially observable and dynamic proved effective in [26]'s demonstration, and displays the usefulness of ABTs for unpredictable environments.

There are some applications of ABTs within the mobile robotics domain that utilize ABTs to handle tasks in multi-robot systems. [11], [29], and [13] all do this to some degree. [11] proposes an algorithm that both automatically generates BTs and allows for online re-planning. They test their proposed framework using three simulations

involving two UAVs on a search and rescue mission. One UAV has a camera to look for the injured, while the other has a first aid kit it takes to the injured. Three different scenarios are tested, one where the environment changes during task execution, one where invariant conditions are never met, and one where a task is not completed in the given time. In each scenario, the BT is automatically altered to adjust to the different challenges. This application involves a partially observable environment as the UAVs only have the one camera to navigate their environment, with no direct knowledge of anything beyond that sensor.

Zhou et al. also uses an ABT for multi-robot task-allocation with a mixture of different kinds of robots. In their case study in [29], they evaluate their ABT framework with a delivery robot, a walking training robot, and a quadrupedal robot. They are placed in a simulated hospital environment and meant to deliver medicine to two specific locations, meet a patient, complete walking training with them, and then take the patient to a specific location. They start in various locations at the start of the simulation, tasks are assigned to them offline, and they each experience some sort of disturbance that hinders their assigned tasks, forcing online re-planning of their behaviors and in some scenarios forcing reallocation of one robot's task to another. This application depends on the ability to observe the full environment surrounding the robot as the ability to reallocate tasks globally means each robot must be able to report its status and the status of the environment around it such as hindrances to its path.

[13] applies ABTs slightly differently than that of other mobile robots. In the paper, Hu et al. use an ABT to make a self-adaptive traffic control model, and tests the model using three types of automatic guided vehicles (AGVs), which must navigate a simu-

lated shop floor and avoid collisions with one another at various intersections. Through the use of an ABT, the AGVs can choose the best strategy for the path they follow while adhering to the traffic rules they are provided with. Their application requires full observability of their shop environment, to know where intersections are and where other AGVs have collided, and they take this information and update their BTs offline based on the information they are given. This offline updating still counts as an ABT due to the BT being updated based on environmental information, but it does differ from previous applications that utilize that information during runtime.

2.1.3 Human-Robot Interaction

Little work has been done for the application of ABTs to human-robot interactions (HRI) despite the prevalence of manual and synthesized BTs in HRI as seen in [12], [10], [7], [21], [19], and [20]. Nonetheless, ABTs do offer benefits to the HRI domain, and this is seen in the paper on a value-driven eldercare robot by Anderson et al. In [2] a simulated and physical Nao robot are placed in an eldercare simulation. The simulation was used to test the framework and then it was applied to the physical robot. The robot's ABT framework allows it to adjust and interrupt tasks, such as if it is charging, but the patient has been stationary for too long, the robot will interrupt the charging task to engage with the patient. Their eldercare robot utilizes a BT software framework proposed in [3] called playful, which allows for online edits to the BT during runtime. Overall the use of an ABT in this case allowed for more reactivity with the patient. While ABTs in HRI are not the most prevalent currently, they could be the next step for some ABTs as seen

in [8] and [24], which says using their ABT framework in the HRI domain would be a possible next step for them.

2.2 Non-Robot Applications

BTs in general began as a means to improve NPC behavior in video games, and they are still being used within the computer game and artificial intelligence (AI) domains. ABTs within the AI domain have been around since around 2015 as seen in [22]. This paper proposes a BT framework that implements learning capabilities to autonomous agents. They test their ABT on firefighting scenarios where the agent's behavior is learned and the BT is further developed by running through the scenarios multiple times. The AI applications of ABTs are unlike the robotic applications, as they do not require in the moment changes to the BT, but rather learn actions based on input from their simulated enviroment. This BT framework can still fall into the category of ABTs soley because the actions are learned from the enviroment.

Chapter 3: Adaptation Methodology

ABTs have been used in a myriad of different applications, and each one goes about making the BT adaptive in different ways. Some methods synthesize BTs from various task planning frameworks and update it during runtime. Some methods place an additional node onto the BT that can learn which actions will be successful and which will not be. There are many different ways researchers have gone about taking in a robot's environmental and internal changes and utilizing a BT to accommodate those changes, and this section looks into those various methods.

Methods	Papers
BT Synthesis from Task Planner	[11] [24] [26] [27]
Reinforcement Learning	[13] [22]
Linear Temporal Logic	[17] [29]
Iterative Expansion of BTs	[4] [8] [18]
Active Inference	[23]

Table 3.1: Adaptive Behavior Tree Methods.

3.1 BT Synthesis from Task Planner

Task planning frameworks for robotics plan possible actions a robot can take based on a given goal conditions. There are various types of task planners and planning algorithms, and their ability to create symbolic actions based on desired goals makes them rather

useful for BT synthesis, as their actions can be outputted in a file that is BT readable, and then they can be used to resynthesize BTs when desired conditions are not met.

The task and motion planning (TAMP) framework is a framework dedicated to finding possible actions for a given task. It is used within the domain of robotic manipulation frequently. Saoji et al. in [25] introduce the TAMP framework and lay out how it deals with the task planning using the Fast Forward planner, the motion planning using the Open Motion Planning Library suite, and The Kautham Project acts as the motion planning server. The different layers of planning are connected via an XML configuration file, and [27] and [24] look to extend this framework into ABTs by having the XML files be formatted like BTs that can then be resynthesized during runtime. The planning tasks are defined using the Planning Domain Definition Language (PDDL) files for the Fast Forward planner. One PDDL file describes the domain while the other describes the problem. Within these files, pre and postconditions are defined as well as information about the environment, initial states, and goal states. The general method [27] goes about making an ABT is through checking the preconditions for an executed behavior, and if it fails, running a recovery procedure to satisfy it, then checking the postcondition of a behavior, and if that fails, the behavior is executed either by a newly implemented action or subtree to satisfy the postcondition. [24] is similar in its use of the TAMP framework, however it uses an ontology-based reasoning framework called Perception and Manipulation Knowledge (PMK), which provides reasonings for sensor related perception, an object's features, spatial evaluation of objects in relation to each other, and for planning to reason the preconditions and action constraints. Ruiz et al. extend the PMK framework by adding knowledge on the available robot actions, and

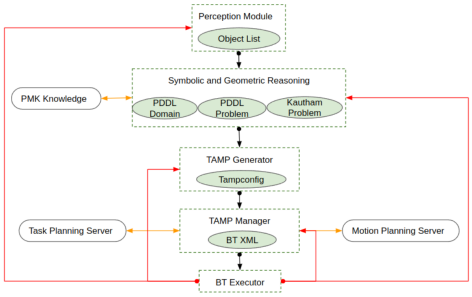

Figure 3.1: TAMP framework from [24]

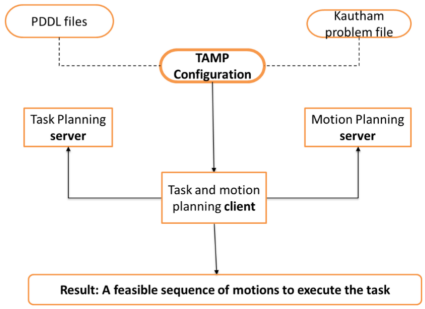

Figure 3.2: TAMP framework from [27]

use this framework to automatically set the PDDL domain and problem files. Both uses of the TAMP framework use a task level BT that handles the pre and postconditions and subtrees to execute the action level plans. At the change in state or failure of a pre or postcondition, the TAMP manager is used to write a new XML file that will then be passed through a BT executor that initializes the BT. In figures 3.1 and 3.2 the two similar TAMP frameworks are shown side by side to display their similarities.

Another task planning framework used to create ABTs is Hierarchical Task Network (HTN) planning. HTN planning is similar to TAMP in that they both have pre and post conditions that must be fulfilled and at the failure of either of those conditions the BT is resynthesized. [26] lays out a three part architecture to their ABT framework composed of the HTN planner, Blackboard, which acts as the link between the planning logic and the real world, and a BT executor. Blackboard holds all of the planning, replanning, and anchoring information. Figure 3.3 shows the layout of the HTN planning framework used by [26].

Another planner used to make ABTs is STRIPS, a classical planning algorithm that can be used to automatically generate BTs. [11] proposes a BT synthesis algorithm

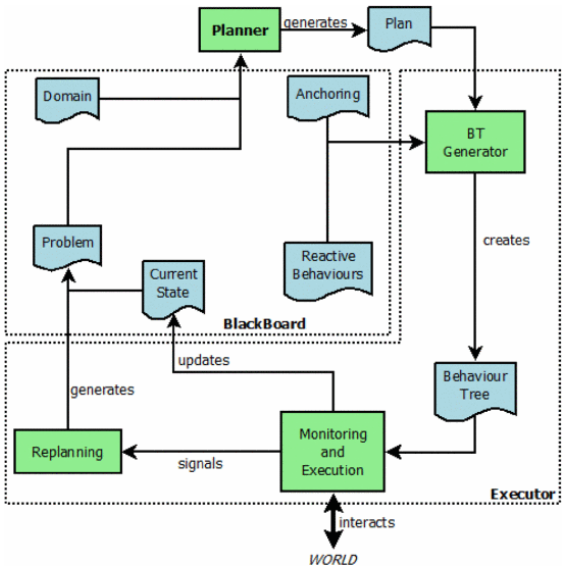

Figure 3.3: HTN BT Generation Architecture from [26]

made up of construction, simulation, and online re-planning. The planner is given a goal and current state, and searches through an action library for an action that can satisfy the goal and find a new one. During simulation, the priority of different actions is determined, so that when the planner seeks a new action it chooses to perform the one of the highest priority. Re-planning occurs when the environment changes or an action fails by having the pre and invariant conditions not be fulfilled. The planner makes those conditions the new goal, and a subtree is sought that can fulfill them.

Overall, the use of task planning frameworks to synthesize BTs and re-synthesize them during runtime based on environmental changes all work fairly similarly in that they use current and goal states or rather pre and post conditions and create a plan from there, and they make BTs adaptive through their changes to the plan based on failure to meet conditions and goals.

3.2 Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning that is used frequently for synthesizing BTs. Q-Learning is a RL algorithm that uses a table of values that can determine which actions to take [9]. An agent uses a function associating states with predetermined rewards, and then the rewards are given to the state-action pairs to improve the estimates on the table of valuable actions. Another RL approach is Hierarchical RL, which utilizes models based on a Semi Markov Decision Process (MDP). Semi MDPs are more general than a standard MDP, which allows for exploration of the time-based aspects of tasks and can split tasks into several subproblems [22]. Pereira et al. utilize the Option framework for their approach to Hierarchical RL. An option is a generlization of an action that calls other options during execution, which creates the idea of a hierarchy wihtin the actions of the MDP. These two approaches of RL go about being implemented into BT frameworks in different ways.

[9] uses Q-Learning to synthesize static BTs, where the Q-values that the algorithm uses to determine a useful action is gathered from simulation. Q-Learning proves itself to be beneficial for synthesizing BTs offline, as it is an approach that can be easily generalized for various applications, and as seen in [13] it can be used to create ABTs as well. The architecture by Hu et al. in [13] uses is meant specifically for AGV control, and they break their approach into three parts, those being a cyber-physical system (CPS) based on a multi-agent system (MAS), a behavior construction model, and a behavior learning model. Figure 3.4 lays out the architecture used for their BT. The CPS based on MAS is the foundation of their architecture, as it deals with the hardware and software in the

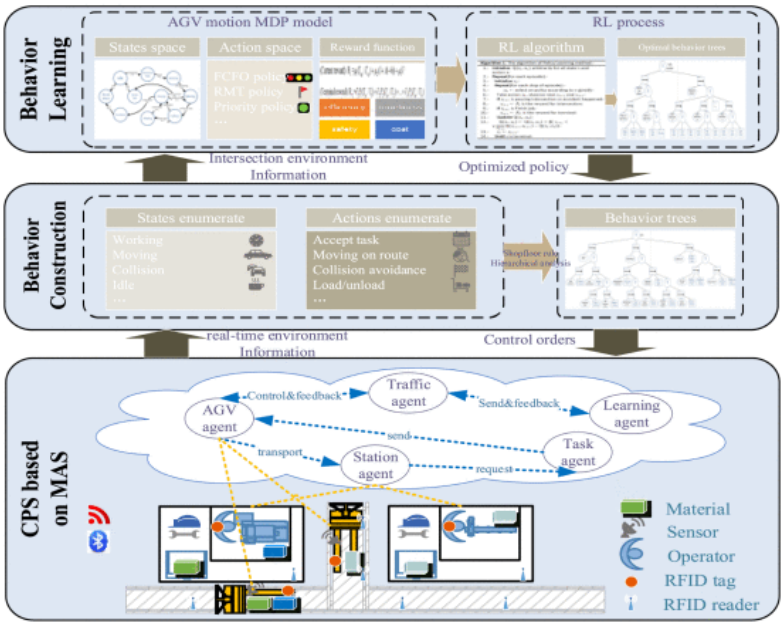

Figure 3.4: BT Generation Architecture for [13]

runtime environment. The behavior construction model creates structures of behavior of the various agents, and develops them into BTs that are used as states and actions in the behavior learning model, which is a key element in the RL process that generates the optimal BT. The behavior learning model portion of this architecture is where the adaptability for the BT comes in, and within this model a Markov Decision Process (MDP) of AGV motion is created and a Q-Learning algorithm takes the MDP model and generates an optimal BT based on the reward values for each behavior. While the paper describes their framework as self-adaptive, it is important to note that no changes happen during runtime as with other ABT frameworks, but rather creates one offline based on real-time information.

Hierarchical RL is used in [22] through the implementation of Learning Nodes to a

BT, embedding RL into the BT itself rather than using RL to generate a BT. There is an action learning node and a composite learning node. For both of these nodes in the learning framework, the behaviors the agents follow are modeled by a human expert. This means the behaviors the agent can execute are manually defined and then learned. For example, the action learning node may be assigned the behavior of grabbing an object, and the node will use RL to learn how to grab different kinds of objects and how to grab them from different positions. This method is less generalizable as it requires expert knowledge on what tasks to train, but it can be considered adaptive in that it can learn how to deal with unknowns.

The usage of RL in ABTs in comparison to the other methods differs greatly, as RL requires learning and training, meaning it is less adaptive during runtime as its tasks are all defined in an offline plan. However, RL for BTs can still fall into the ABT umbrella as the actions that the agent learns and accounts for can still be based on real world, real time information, it is just that the plan will not adapt to a sudden change to that information.

3.3 Linear Temporal Logic

Linear Temporal Logic (LTL) is a tool that can be used to specify the goal properties of a system [17]. It is frequently used for encoding time-specific tasks and can automatically synthesize the time specifications to the transition behaviors of a system [29]. Zhou et al. use figure 3.5 to show how their LTL framework is layed out. It displays the high level LTL task planning, and at the creation of a product automation, it is sent

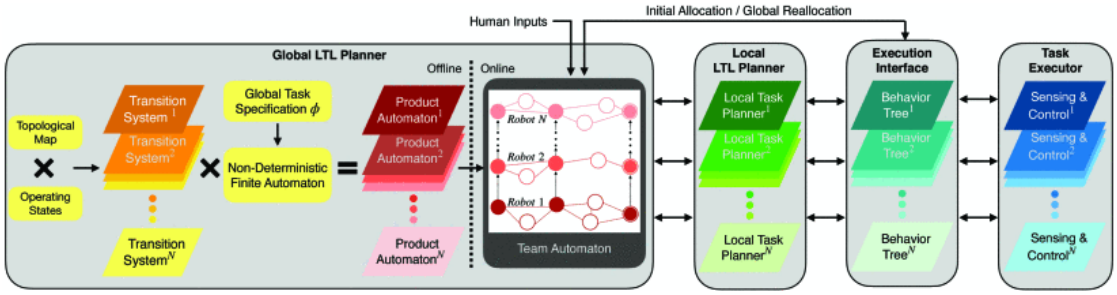

Figure 3.5: The planning framework of [29]

to a team automaton and actions are assigned to individual robots in a system offline. During runtime, the local task planner, BT, and robot hardware are meant to react to disturbances such as loss of balance, environmental changes, etc., and react accordingly by reallocating actions. Rather than synthesizing a new BT at environmental state changes, [29] chooses to reallocate tasks locally on a single agent and globally within the entire multi-robot system based on the kind of failure it encounters. If a single agent is deemed incapable of completing a task, the tasks are reallocated to the other robots in the system, but if an unexpected change in the robot or environment occurs, the planner locally re-allocates tasks by finding a new path for it to follow based on the sensed information on the environment. Overall, the use of LTL for task reallocation allows for not only a single robot to adapt to a change in its environment, but for all the robots in a system to communicate with each other about environmental changes and react accordingly.

Another use of LTL for ABTs is to use them in combination with classical planning as seen in [17]. Lan et al. use a classical planning algorithm for LTL synthesis and iterative planning to approach the reactive synthesis problem. A BT acts as the intermediate interface that executes the plan, receives feedback on action execution, and facilitates the replanning process. The execution for each action starts with checking the precon-

ditions, performing the actual task, and updating the BT on the effects of the task. In the event of an action failure or environmental change, the high-level task planner replans a new action. In [17] the BT itself is updated via the LTL task planning framework in the event of environmental changes.

3.4 Iterative Expansion of BTs

Another method of creating ABTs is through iteratively expanding a BT based on environmental changes. Both [18] and [4] resolve the issue of unpredictable environments through dynamic expansion of their BTs. [18] uses what Li et al. refer to as a reactive BT, which dynamically expands based on actions and predicates. [18] proposes a reactive BT that deals with partially observable environments through dynamic rollbacks that repair sequential dependence. A partially observable MDP describes the task scene and a PDDL-like language describes the planning problem. To make the reactive BT work in partially observable environments, the paper focuses on three key aspects. The first aspect is dynamic expansion, which occurs when a condition node in the BT returns failure. The BT expands according to the last failed node by replacing the failed node with a subtree whose postcondition has the predicate of the failed node. Another aspect is priority conflict, which handels if the added subtree conflicts with a conditional that already returned success. It handles this conflict by increasing the priority of the new subtree, placing it below the sequence node and in front of the conflicted nodes. The third aspect to consider when making the reactive BT is dependence conflicts. If a subtree being added depends on a condition to be true, but can not presently perceive

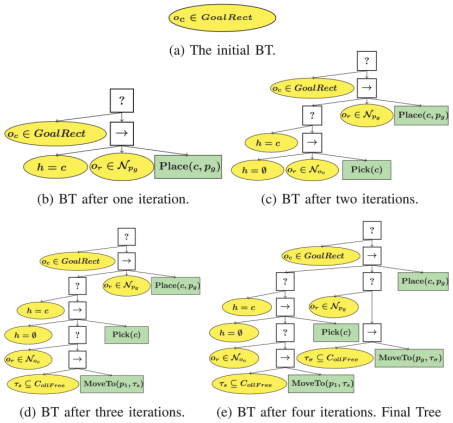

Figure 3.6: BT expansion from [4]

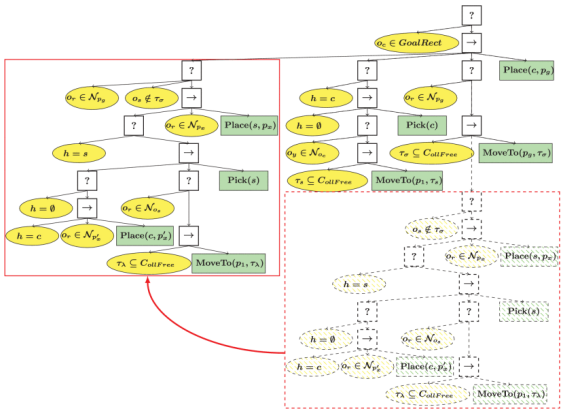

Figure 3.7: Conflict Resolution for subtrees from [4]

that condition, a subtree collapsing method is called to solve this conflict. The subtree collapsing method collapses the subtree to the original condition node and moves it to its original position.

[4] is similar to [18] in its approach to expand a BT with subtrees when conditions are not met or fail, however Colledanchise et al. base their method off of backchaining, which is to say they start with the desired goal and work backwards from it. The BT starts with a goal condition node, and iteratively select actions that will achieve that goal, and if those actions have unmet preconditions, those actions are extended with actions that will meet them. The paper refers to the subtrees that are added into the main BT as atomic BTs, and the first step of the BT algorithm converts a list of actions into a list of atomic BTs. The list of atomic BTs is then used to iteratively build a robust BT from a BT with minimal nodes. The BTs in figure 3.6, show the expansion process, and the BT in figure 3.7 shows how [4] deals with conflicts in priority similarly to [18] by increasing the priority of the new subtree.

Alongside expansion of a BT, some ABT methods reconfigure the BT, both adding and removing nodes based on changes around or within the robot. [8] proposes a BT it calls the Reconfigurable Behavior Tree, which is meant to utilize the modularity of the standard BT by using the defined tree structure that can be reconfigured during run-time. Reconfiguration in this case looks like the addition or removal of branches to the tree based on sensed information to the environment. Their BT contains the standard components of a BT, as well as a long-term memory (LTM), a working memory (WM), a priority handler called the Emphasizer, and an Instantiator. The instantiatior accesses the LTM and the WM, taking a subtask from the LTM and placing it in a corresponding subtree from the WM. It is through the instantiator that the reconfigurable BT has its reconfiguration abilities as it can dynamically load and instantiate subtrees. If new sensory data is obtained or the postcondition of subtree is changed, the emphasizer recalculates priority and sets a flag that denotes the change, which triggers the instantiator to deallocate the current subtree and instantiate the subtree of highest priority.

3.5 Active Inference

Another method of creating an ABT is through the combination of active inference and BTs. Active inference is an idea that comes from neuroscience that describes the brain's method of perceiving an environment and acting in accordance to it [23]. This idea has recently shown potential in control engineering and robotics, using this neuroscientific idea to describe an algorithm for perceiving, planning, acting, and learning based on biological concepts [23]. Active Inference can be used to solve complex problems when

given a context sensitive prior, or rather the desired state for the robot to reach, through which assumptions can be made about the world's state using a Bayesian interface [23]. The combination of active inference and BTs is seen in [23], where a BT acts as a prior, allowing for global reactivity in predicted situations. The desired states are determined offline, and the action selection happens online by the active inference scheme, which allows for local reactivity in unpredictable situations. The BT is used as a prior through the introduction of a new node called a prior node. This node is similar to an action node, but rather than causing an action to happen it sets the desired value of a state and runs through the active interference scheme to select an action that will match its current state to the desired one. The combination of active inference and behavior trees create an ABT, because the BT is choosing actions online to execute based on sensed information, allowing for it to adjust to unpredictable environments.

Chapter 4: Gaps in Literature

ABTs have a multitude of applications currently, but they are lacking in fields other than manipulators and mobile robots. ABTs are currently being used for terrain traversal and pick and place object manipulation in unpredictable environments more than anything else. These applications show how ABTs currently deal with low-level, simple tasks, and while papers like [11], [8], [23], [27], and [24] refer to utilizing ABTs to manage more complex behavior in future works, it is important to note that little is done in that realm as of now. It is also important to note in reference to applications that little work with ABTs has been done with ABTs outside of manipulators and mobile robotics. The HRI domain utilizes BTs already for applications such as child-robot interactions as seen in [12] and [7], and human-robot teaming as seen in [19], [20], [21], and [10]. As human behavior is rather unpredictable and BTs are already being applied in this field, the utilization of an ABT of some sort is a logical next step for this domain. Another domain where more work can be done is within multi-robot applications of ABTs. ABTs can be beneficial to multi-robot applications as they allow for tasks to be reallocated throughout the multi-robot system, so that the robots can communicate with one another about the parts of the environment they sense and adjust for changes on both a local and global level. ABTs for multi-robot systems are being applied in multi-robot search and rescue simulations ([11]), heterogeneous multi-robot task allocation ([29]), and traffic control for multiple AGVs ([13]), but there is room for more work to be done within

this domain. ABTs are already seen to be beneficial to these applications, allowing for the robots within these multi-robot systems to communicate with one another and adjust their tasks locally and globally based on sensed information, and ABTs could be applied further to incorporate reactive multi-robot systems in new domains and more complex, reactive actions for the individual robots within the system. Additionally, many of the current ABT applications depend on full observability due to the need to know the current state of both the robot, the surrounding environment, and possibly of other robots in a system, and while some papers focus on working in partially observable environments, [11], [26], [15], [23], [18], the majority of applications do not handle robots that cannot obtain all of this information all at once.

There is also a matter of the methodology used to create ABTs, of which there are many. All of the different ABT methods either resynthesize a BT at the failure of reaching a goal or a state change, implement nodes into the BT that can learn from failure or intake environmental information , or replace a failed node with a subtree. All of these methods have their own benefits and downfalls, which are application-specific as to if they are a generally good method for creating an ABT. The gaps within ABT methodology are more to do with what has not been utilized, as in there are many ABT generation methods that could be made to be adaptive but are not currently, alongside the dependency these methods have on simulations. In terms of what has not been utilized, there are learning methods such as learning from demonstration, evolution-inspired learning, and case-based reasoning that have been used in BT creation according to [14], but none have been utilized for making a BT adaptive. All these methods have the capabilities to be used in an ABT framework, as they can take inspiration from synthesizing and updat-

ing BTs from the various learning algorithms or use learning nodes that use a learning method apart from RL. There is also the fact that there are many different ways people have gone about synthesizing a BT such as the methods mentioned in [14], [28], [1], and [16] that could likely be applied to a resynthesis processed such as the ones used in [27] and [24], though with a different planning framework or different method of BT synthesis altogether. In regards to the dependence on simulation, as of now many of these methods are validating using simulations with simple tasks for the agents to complete, meaning the next step for many is to implement their ABT method on a physical machine with more complicated goals, which will serve to further validate their various ABT methods. However another future goal with ABTs is to have a robot that can adapt its tasks in a space without the use of simulation. This would be beneficial in domains that cannot be easily simulated, like in the majority of HRI. Overall, the gaps within methodology have to do primarily with the fact that BTs have been created in a manner of different ways, all of which could likely be made to be adaptable, and with the fact that there is a current heavy dependence on simulated environments to validate BT methods.

Chapter 5: Conclusion

In this survey, we look at various applications and methods of generation of adaptive behavior trees (ABTs). ABTs are BTs that receive information about changes in an agent's environment, whether that be a change in the state of the agent, a sudden obstacle in the agent's path, or an object of interest being in an unexpected location, and adjust the tasks the agent performs via resynthesizing a BT based on sensed information, replacing failed execution nodes with subtrees, or using nodes in a BT that can learn from the sensed information or trigger a replanning process. There are many different ways people have gone about applying ABTs to various domains, from mobile robots to AI agents, all of which have the primary goal of being able to react to dynamic, unpredictable changes. The ability to react is implemented via synthesizing a BT from a task planning framework, utilizing RL, expanding BTs at the failure of a task, or implementing unique nodes that can update the BT in some way. There are many different methods of creating ABTs and various applications of them, but there is still space to expand upon the ABT idea to generate them in new ways , apply them to more domains, and create more complex, dynamic behaviors.

Bibliography

[1] Faseeh Ahmad, Matthias Mayr, and Volker Krueger. Learning to adapt the parameters of behavior trees and motion generators to task variations. *arXiv preprint arXiv:2303.08209*, 2023.

[2] Michael Anderson, Susan Leigh Anderson, and Vincent Berenz. A value-driven eldercare robot: Virtual and physical instantiations of a case-supported principle-based behavior paradigm. *Proceedings of the IEEE*, 107(3):526–540, 2019.

[3] Vincent Berenz and Stefan Schaal. The playful software platform: Reactive programming for orchestrating robotic behavior. *IEEE Robotics & Automation Magazine*, 25(3):49–60, 2018.

[4] Michele Colledanchise, Diogo Almeida, and Petter Ögren. Towards blended reactive planning and acting using behavior trees. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8839–8845, 2019.

[5] Michele Colledanchise, Alejandro Marzinotto, Dimos V. Dimarogonas, and Petter Oegren. The advantages of using behavior trees in mult-robot systems. In *Proceedings of ISR 2016: 47st International Symposium on Robotics*, pages 1–8, 2016.

[6] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. CRC Press, jul 2018.

[7] Enrique Coronado, Xela Indurkhya, and Gentiane Venture. Robots meet children, development of semi-autonomous control systems for children-robot interaction in the wild. In *2019 IEEE 4th International Conference on Advanced Robotics and Mechatronics (ICARM)*, pages 360–365, 2019.

[8] Pilar de la Cruz, Justus Piater, and Matteo Saveriano. Reconfigurable behavior trees: Towards an executive framework meeting high-level decision making and control layer features. In *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 1915–1922, 2020.

[9] Rahul Dey and Chris Child. Ql-bt: Enhancing behaviour tree design and implementation with q-learning. In *2013 IEEE Conference on Computational Inteligence in Games (CIG)*, pages 1–8, 2013.

[10] Fabio Fusaro, Edoardo Lamon, Elena De Momi, and Arash Ajoudani. A human-aware method to plan complex cooperative and autonomous tasks using behavior trees. In *2020 IEEE-RAS 20th International Conference on Humanoid Robots (Humanoids)*, pages 522–529, 2021.

[11] Chuanxiang Gao, Yu Zhai, Biao Wang, and Ben M. Chen. Synthesis and online re-planning framework for time-constrained behavior tree. In *2021 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, pages 1896–1901, 2021.

[12] Ameer Helmi, Emily Scheide, Tze-Hsuan Wang, Samuel W. Logan, Geoffrey A. Hollinger, and Naomi T. Fitter. Gobot An autonomous assistive robot using behavior trees to encourage child mobility. *Association for Computing Machinery*, 2023.

[13] Hao Hu, Xiaoliang Jia, Kuo Liu, and Bingyang Sun. Self-adaptive traffic control model with behavior trees and reinforcement learning for agv in industry 4.0. *IEEE Transactions on Industrial Informatics*, 17(12):7968–7979, 2021.

[14] Matteo Iovino, Edvards Scukins, Jonathan Styrud, Petter Ögren, and Christian Smith. A survey of behavior trees in robotics and ai. *Robotics and Autonomous Systems*, 154:104096, 2022.

[15] Matteo Iovino, Jonathan Styrud, Pietro Falco, and Christian Smith. Learning behavior trees with genetic programming in unpredictable environments. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4591–4597, 2021.

[16] Matteo Iovino, Jonathan Styrud, Pietro Falco, and Christian Smith. Learning behavior trees with genetic programming in unpredictable environments. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4591–4597, 2021.

[17] Menglu Lan, Shupeng Lai, Tong Heng Lee, and Ben M. Chen. Autonomous task planning and acting for micro aerial vehicles. In *2019 IEEE 15th International Conference on Control and Automation (ICCA)*, pages 738–745, 2019.

[18] Ning Li, Hao Jiang, Chunpeng Li, and Zhaoqi Wang. Towards adaptive behavior trees for robot task planning. In *2022 China Automation Congress (CAC)*, pages 6720–6725, 2022.

[19] Chris Paxton, Andrew Hundt, Felix Jonathan, Kelleher Guerin, and Gregory D. Hager. Costar: Instructing collaborative robots with behavior trees and vision. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 564–571, 2017.

[20] Chris Paxton, Andrew Hundt, Felix Jonathan, Kelleher Guerin, and Gregory D. Hager. User experience of the costar system for instruction of collaborative robots. 2017.

[21] Chris Paxton, Felix Jonathan, Andrew Hundt, Bilge Mutlu, and Gregory D. Hager. Evaluating methods for end-user creation of robot task plans. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6086–6092, 2018.

[22] Renato de Pontes Pereira and Paulo Martins Engel. A framework for constrained and adaptive behavior-based agents. *arXiv preprint arXiv:1506.02312*, 2015.

[23] Corrado Pezzato, Carlos Hernández Corbato, Stefan Bonhof, and Martijn Wisse. Active inference and behavior trees for reactive action planning and execution in robotics. *IEEE Transactions on Robotics*, 39(2):1050–1069, 2023.

[24] Oriol Ruiz-Celada, Parikshit Verma, Mohammed Diab, and Jan Rosell. Automating adaptive execution behaviors for robot manipulation. *IEEE Access*, 10:123489–123497, 2022.

[25] Siddhant Saoji and Jan Rosell. Flexibly configuring task and motion planning problems for mobile manipulators. In *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, volume 1, pages 1285–1288, 2020.

[26] José Ángel Segura-Muros and Juan Fernández-Olivares. Integration of an automated hierarchical task planner in ros using behaviour trees. In *2017 6th International Conference on Space Mission Challenges for Information Technology (SMC-IT)*, pages 20–25, 2017.

[27] Parikshit Verma, Mohammed Diab, and Jan Rosell. Automatic generation of behavior trees for the execution of robotic manipulation tasks. In *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–4, 2021.

[28] Wendi Wu, Jinghua Li, Yunlong Wu, Xiaoguang Ren, and Yuhua Tang. Multi-uav adaptive path planning in complex environment based on behavior tree. In *International Conference on Collaborative Computing: Networking, Applications and Worksharing*, pages 494–505. Springer, 2020.

[29] Ziyi Zhou, Dong Jae Lee, Yuki Yoshinaga, Stephen Balakirsky, Dejun Guo, and Ye Zhao. Reactive task allocation and planning for quadrupedal and wheeled robot teaming. In *2022 IEEE 18th International Conference on Automation Science and Engineering (CASE)*, pages 2110–2117, 2022.